ESTRUCTURAS DE DATOS - x

Escuela de Ingeniería de Ciencias y Sistemas
Facultad de Ingeniería
Universidad de San Carlos de Guatemala

Día, fecha: xx xx xxxx
Hora de inicio: 17:20
Auxiliar: [por completar]
Semana: [por confirmar]

MATRICES DISPERSAS

Unidad 4

Tema: Matriz Dispersa

Definición, características y operaciones
Escuela de Ingeniería de Ciencias y Sistemas
Universidad de San Carlos de Guatemala

Anuncios

xxxx
xxxx
xxxx
xxxx

Completar con los avisos oficiales del curso cuando estén disponibles.

Valor: Solidaridad, 5 minutos.
Área de conocimiento: 55 minutos.

AGENDA

Primera parte

Solidaridad: 5 min.
Definición y ahorro de memoria: 7 min.
Formatos de representación: 12 min.
Operaciones básicas: 10 min.

Segunda parte

Operaciones matriciales: 12 min.
Aplicaciones y ejercicio: 9 min.
Dudas y cierre: 5 min.

Conocimiento: 55 min.
Sesión completa: 60 min.

Valor de la semana: Solidaridad

Descripción y relación con la sesión

Apoyar a los demás, especialmente en momentos difíciles.

Durante el estudio de matrices dispersas, se demuestra al explicar con paciencia una representación, compartir estrategias y ayudar a detectar errores sin sustituir el trabajo del compañero.

Aplicación del valor

Actividad de 5 minutos:
1 min: leer y explicar el valor.
3 min: en parejas, explicar una tripleta y revisar sus índices con respeto.
1 min: compartir cómo el apoyo ayudó a comprender.

Evidencia: ambos pueden explicar el procedimiento.

Competencias a desarrollar

Competencia 1

Representar matrices con predominio de ceros mediante estructuras dispersas, justificando la elección del formato según el uso de memoria y las operaciones requeridas.

Competencia 2

Implementar y verificar inserción, búsqueda, actualización, eliminación, transposición, suma y producto matriz-vector, aplicando índices y enlaces consistentes.

2.1 Contenido

No.	Unidad	Tema	Sub Tema
1	4	Matriz Dispersa	Definición y características de una matriz dispersa
2	4	Matriz Dispersa	Operaciones de una matriz dispersa
3	4	Matriz Dispersa	Representaciones: tripletas, COO, CSR y CSC
4	4	Matriz Dispersa	Listas enlazadas y estructura ortogonal
5	4	Matriz Dispersa	Inserción, búsqueda, actualización y eliminación
6	4	Matriz Dispersa	Transposición, suma y multiplicación
7	4	Matriz Dispersa	Complejidad, aplicaciones y ejercicios prácticos

MATRICES DISPERSAS

Guardar lo significativo

Estructuras que evitan almacenar explícitamente la mayoría de los ceros

1. Del arreglo bidimensional a la dispersión

Fundamento

Una matriz densa reserva una posición para cada combinación de fila y columna. Esa representación es sencilla, pero puede desperdiciar memoria cuando casi todas las entradas son cero.

Una representación dispersa guarda los valores relevantes y la información necesaria para localizar sus coordenadas.

Ejemplo y aplicación

Matriz de 1,000 × 1,000:
1,000,000 de posiciones posibles.

Si solo 2,000 contienen valores no nulos, almacenar todas las posiciones puede ser costoso. La alternativa dispersa debe conservar también las dimensiones y los índices de los elementos registrados.

2. Definición de matriz dispersa

Fundamento

Una matriz se considera dispersa cuando tiene suficientes ceros como para que una representación especializada resulte conveniente. No existe un porcentaje universal que defina todos los casos.

La decisión depende del patrón de valores, del costo de almacenar índices y de las operaciones que se realizarán.

Ejemplo y aplicación

Una matriz con muchos ceros puede aparecer al representar conexiones de un grafo, interacciones entre usuarios y productos o relaciones entre palabras y documentos.

La estructura matemática sigue siendo una matriz: cambia su forma de almacenamiento, no el significado de sus entradas.

3. Dimensiones, coordenadas y notación

Fundamento

Usaremos r filas y c columnas. Una entrada se escribe A[i,j]. En los ejemplos, los índices comienzan en cero.

Los índices válidos cumplen:
$0 \le i < r$; $0 \le j < c$.

Las dimensiones se conservan aunque existan filas o columnas completamente vacías.

Ejemplo y aplicación

Ejemplo base A, de 4 × 5:

```
[ 0  0  5  0  0 ]
[ 8  0  0  0  0 ]
[ 0  0  0 -2  0 ]
[ 0  6  0  0  9 ]
```

A[0,2] = 5; A[2,3] = -2.
A[0,0] = 0 implícito.

4. Cantidad de no nulos y densidad

Fundamento

Sea z = nnz(A), la cantidad matemática de entradas no nulas.

Densidad = z / (r × c).
Proporción de ceros = 1 − densidad.

Estas fórmulas suponen r × c > 0. Una matriz vacía requiere tratar el caso por separado.

Ejemplo y aplicación

En A hay 5 valores no nulos y 20 posiciones.

Densidad = 5/20 = 0.25 = 25 %.
Proporción de ceros = 75 %.

En algunas bibliotecas, nnz cuenta entradas almacenadas, incluidos ceros explícitos. Aquí distinguiremos los no nulos de la cantidad almacenada.

5. Ceros implícitos y ceros almacenados

Fundamento

En una representación dispersa, una coordenada que no está registrada suele interpretarse como cero. Eso evita crear una entrada por cada cero.

Algunas bibliotecas permiten guardar ceros explícitos. Estos ocupan memoria y pueden permanecer después de una operación aritmética.

Ejemplo y aplicación

Convención del material:
• Se conserva como máximo una entrada por coordenada.
• No se almacenan ceros explícitos.
• Asignar cero elimina una entrada existente.

Si otra implementación permite duplicados o ceros, deberá definir cómo normalizarlos.

6. Memoria densa frente a memoria dispersa

Representación	Contenido	Memoria aproximada	A de 4 × 5
Densa	r × c valores	r·c·b	160 bytes
COO	Valor + fila + columna	z·(b + 2s)	80 bytes
CSR	Valor + columna + punteros	z·(b + s) + (r+1)·s	80 bytes
CSC	Valor + fila + punteros	z·(b + s) + (c+1)·s	84 bytes

Supuestos: b = 8 bytes por valor, s = 4 por índice y z = 5. No incluye metadatos, alineación ni sobrecarga de objetos.

7. Cuándo conviene una representación dispersa

Fundamento

Ahorrar valores no basta: es necesario compensar el espacio de los índices y la complejidad adicional.

Con COO y los tamaños del ejemplo, cada entrada utiliza 16 bytes. La representación densa utiliza 8 bytes por posición. COO ahorra memoria de datos si $16z < 8rc$.

Ejemplo y aplicación

Bajo esos supuestos:
$z < rc/2$.

Este umbral solo compara el almacenamiento indicado; no garantiza mayor velocidad. Si la matriz se vuelve suficientemente densa, el arreglo completo puede ser más simple y rápido gracias a su acceso directo y localidad.

8. Representación mediante tripletas

Fundamento

Una tripleta guarda fila, columna y valor de una entrada no nula. Las dimensiones y la cantidad de entradas se mantienen como metadatos.

Algunos materiales incluyen una tripleta inicial (filas, columnas, cantidad). Esa cabecera describe la matriz y no representa un valor numérico de ella.

Ejemplo y aplicación

```
Ejemplo base:
Cabecera: (4,5,5).
Entradas:
(0,2,5)
(1,0,8)
(2,3,-2)
(3,1,6)
(3,4,9)

Total: 5 entradas, más la cabecera.
```

9. Formato COO: coordenadas en arreglos

Fundamento

COO almacena tres arreglos paralelos: filas, columnas y valores. Las entradas con el mismo índice de arreglo describen una misma coordenada.

Es útil para construir matrices a partir de una secuencia de contribuciones. Puede normalizarse ordenando y agrupando coordenadas repetidas.

Ejemplo y aplicación

```
Para A:
row = [0, 1, 2, 3, 3]
col = [2, 0, 3, 1, 4]
val = [5, 8, -2, 6, 9]
shape = (4, 5)

La entrada k = 2 representa
A[2,3] = -2.
No se debe desalinear un arreglo.
```

10. Orden y coordenadas duplicadas

Fundamento

Ordenar por fila y luego por columna facilita recorridos y fusiones. COO no exige siempre ese orden para existir.

Una coordenada repetida necesita una política: asignación con reemplazo o acumulación de contribuciones. Ambas son válidas, pero representan operaciones diferentes.

Ejemplo y aplicación

Contribuciones (1,0,3) y (1,0,5): Ensamblar por suma produce (1,0,8).

Asignar 3 y después asignar 5 produce (1,0,5).

En nuestras operaciones de modificación usaremos asignación. Para construir desde contribuciones, se sumarán duplicados y se retirarán los ceros resultantes.

11. Formato CSR: compresión por filas

Fundamento

CSR agrupa las entradas por fila. Utiliza valores, índices de columna y un arreglo row_ptr que delimita cada fila.

row_ptr tiene r + 1 posiciones. La fila i ocupa desde row_ptr[i] hasta row_ptr[i+1], sin incluir ese último límite.

Ejemplo y aplicación

```
Para A:
val = [5, 8, -2, 6, 9]
col = [2, 0, 3, 1, 4]
row_ptr = [0, 1, 2, 3, 5]

Fila 3: intervalo [3,5).
Sus entradas son val[3] = 6 en columna 1
y val[4] = 9 en columna 4.
```

12. Lectura e invariantes de row_ptr

Fundamento

Con índices base cero:
row_ptr[0] = 0.
row_ptr[r] = cantidad almacenada.
Los punteros no disminuyen.

La resta row_ptr[i+1] − row_ptr[i] indica cuántas entradas tiene la fila i. Dos punteros consecutivos iguales identifican una fila vacía.

Ejemplo y aplicación

Ejemplo: row_ptr = [0,1,1,3].

La matriz tiene 3 filas.
Fila 0: una entrada.
Fila 1: cero entradas.
Fila 2: dos entradas.

La fila vacía no desaparece y mantiene su posición dentro de las dimensiones de la matriz.

13. Conversión de COO a CSR

Fundamento

Se cuentan las entradas por fila y se calculan sumas prefijas para construir row_ptr. Después se distribuyen valores y columnas en sus segmentos.

Si se necesita CSR canónico, también se ordenan las columnas dentro de cada fila y se consolidan coordenadas repetidas.

Ejemplo y aplicación

En A, conteos por fila:
[1, 1, 1, 2].

Sumas prefijas, iniciando en cero:
[0, 1, 2, 3, 5].

Este último arreglo es row_ptr. El costo de ordenar o consolidar se analiza aparte; no debe asumirse que COO ya está normalizado.

14. Formato CSC: compresión por columnas

Fundamento

CSC aplica la misma idea de CSR, pero agrupa las entradas por columna. Mantiene valores, índices de fila y col_ptr de longitud c + 1.

Es apropiado para recorrer columnas. Elegir CSR o CSC depende del acceso predominante y de los algoritmos posteriores.

Ejemplo y aplicación

```
Para A, agrupada por columna:
val = [8, 6, 5, -2, 9]
row = [1, 3, 0, 2, 3]
col_ptr = [0,1,2,3,4,5]

Columna 2: intervalo [2,3).
Contiene A[0,2] = 5.
```

15. Comparación de formatos dispersos

Formato	Organización	Ventaja	Limitación
Tripletas / COO	Coordenadas explícitas	Construcción sencilla	Búsqueda sin índice
CSR	Entradas por fila	Recorridos por fila	Cambio estructural costoso
CSC	Entradas por columna	Recorridos por columna	Cambio estructural costoso
Ortogonal	Enlaces por fila y columna	Modificación y recorridos	Sobrecarga de punteros

Ningún formato es mejor para todas las operaciones. Una aplicación puede construir en un formato y calcular en otro.

16. Elegir el formato según la operación

Fundamento

La construcción y el cálculo no siempre requieren la misma organización. Una aplicación puede recopilar contribuciones en COO o modificar listas y después convertir a CSR para realizar muchos productos matriz-vector.

El costo de convertir debe compararse con el trabajo que se ahorrará en las operaciones posteriores.

Ejemplo y aplicación

Preguntas para decidir:
- ¿Habrá cambios frecuentes de coordenadas?
- ¿Predominan filas o columnas?
- ¿Se repetirán productos numéricos?
- ¿Cuánto cuestan los índices o punteros?

Para una matriz pequeña, la representación densa puede ser suficiente y más sencilla.

17. Listas enlazadas por filas

Fundamento

Otra representación mantiene una lista de entradas por cada fila. Cada nodo registra columna, valor y siguiente. Las cabeceras permiten localizar el inicio de cada lista.

Si las listas están ordenadas por columna, se puede detener una búsqueda cuando se supera la columna solicitada.

Ejemplo y aplicación

Fila 3 de A:
Cabecera → (col 1, val 6) → (col 4, val 9).

Una fila vacía tiene una cabecera sin nodos de datos.

Los enlaces permiten insertar sin desplazar un arreglo completo, pero cada nodo consume memoria adicional y debe localizarse su posición.

18. Matriz ortogonal: enlaces en dos direcciones

Fundamento

Una matriz ortogonal enlaza cada nodo de datos en su fila y en su columna. El mismo nodo pertenece a ambos recorridos.

Puede utilizar listas simples o dobles; por eso no debe suponerse que siempre hay cuatro punteros por nodo. Aquí usaremos derecha y abajo, más cabeceras.

Ejemplo y aplicación

Nodo de ejemplo:
(fila = 3, columna = 1, valor = 6).

Derecha: siguiente entrada de la fila 3.
Abajo: siguiente entrada de la columna 1.

Las cabeceras de filas y columnas conservan acceso a ambas organizaciones sin duplicar el valor almacenado.

19. Invariantes de la estructura ortogonal

Fundamento

Cada entrada no nula aparece una sola vez como nodo de datos y es alcanzable desde su cabecera de fila y de columna.

Los recorridos por fila se ordenan por columna; los de columna, por fila. No deben existir enlaces a memoria liberada ni nodos presentes en una sola dirección.

Ejemplo y aplicación

Antes de modificar:
- Validar los índices.
- Definir si la coordenada existe.
- Localizar los predecesores.

Después de modificar:
- Conservar el orden.
- Actualizar ambos enlaces.
- Ajustar el contador si cambia la cantidad de nodos.

20. Consulta de una entrada

Fundamento

Consultar A[i,j] significa recuperar su valor si está almacenado o devolver cero si la coordenada válida no aparece.

El costo depende del formato. En una lista ordenada se recorre la fila; en CSR con columnas ordenadas puede utilizarse búsqueda binaria dentro de ese segmento.

Ejemplo y aplicación

Buscar A[3,4] en A:
Fila 3 contiene columnas 1 y 4.
Se encuentra el valor 9.

Buscar A[3,2]:
No existe una entrada en columna 2.
Resultado: cero implícito.

Consultar una coordenada fuera de rango es un error, no un cero implícito.

21. Inserción de un valor no nulo

Fundamento

Para insertar un valor en una coordenada ausente se valida el rango y se crea la entrada en el lugar apropiado.

En una matriz ortogonal debe enlazarse el mismo nodo tanto en la fila como en la columna. En arreglos comprimidos, una inserción estructural puede exigir desplazar datos y ajustar punteros.

Ejemplo y aplicación

Insertar A[1,4] = 7 en el ejemplo base.

La fila 1 pasa de {(0,8)} a {(0,8),(4,7)}.
La columna 4 contiene ahora filas 1 y 3.

nnz aumenta de 5 a 6. Las dimensiones permanecen 4 × 5.

22. Actualización y asignación de cero

Fundamento

Si la coordenada ya existe, asignar otro valor no nulo modifica su contenido y mantiene la cantidad de entradas.

En la convención del material, asignar cero retira la entrada. No es necesario guardar un nodo cuyo valor sea cero.

Ejemplo y aplicación

Partiendo del ejemplo base:
A[0,2] = 12 reemplaza el 5.
nnz sigue siendo 5.

A[2,3] = 0 elimina el -2.
nnz baja a 4.

Asignar cero a una coordenada ya ausente no crea entradas ni cambia el tamaño.

23. Eliminación en una matriz ortogonal

Fundamento

Eliminar requiere desconectar el nodo de dos listas. Primero se localizan el nodo y sus enlaces anteriores; luego se actualizan el recorrido horizontal y el vertical.

La memoria se libera cuando ya no existe ningún enlace activo hacia ese nodo.

Ejemplo y aplicación

Eliminar A[3,1] = 6.

La fila 3 conserva únicamente la columna 4.
La columna 1 queda vacía.
El nodo eliminado deja de ser alcanzable por ambas cabeceras.

Error frecuente: quitarlo solo de la fila y dejar una referencia inválida desde la columna.

24. Pseudocódigo de asignación

Fundamento

Una única operación set permite expresar inserción, actualización y eliminación. Su comportamiento debe estar definido para los casos de existencia y ausencia.

El pseudocódigo abstrae cómo se localiza la entrada; cada representación implementa esa búsqueda de forma distinta.

Pseudocódigo

```
set(i,j,v):
  validar_indices(i,j)
  nodo = buscar(i,j)
  si nodo existe:
    si v == 0: eliminar(nodo)
    si no: nodo.valor = v
  si no existe y v != 0:
    insertar(i,j,v)

Actualizar nnz solo al crear o retirar.
```

25. Recorridos y reconstrucción densa

Fundamento

Un recorrido disperso visita únicamente las entradas almacenadas, más las cabeceras o punteros necesarios.

Reconstruir una matriz densa requiere crear r × c posiciones inicializadas en cero y después escribir los valores registrados. Ambas tareas tienen costos diferentes.

Ejemplo y aplicación

Para visualizar A:
1. Crear una matriz 4 × 5 con ceros.
2. Recorrer sus 5 entradas.
3. Escribir cada valor en su coordenada.

Listar tripletas puede ahorrar trabajo; imprimir las 20 celdas exige producir también los ceros implícitos.

26. Transposición: intercambiar filas y columnas

Fundamento

La transpuesta $T = A^T$ intercambia coordenadas: $T[j,i] = A[i,j]$. Las dimensiones $r \times c$ pasan a $c \times r$.

En COO basta intercambiar los arreglos de fila y columna para representar la transpuesta; si se exige orden por fila, puede ser necesario reordenar.

Ejemplo y aplicación

A es 4×5; A^T es 5×4.

```
(0,2,5) → (2,0,5)
(1,0,8) → (0,1,8)
(2,3,-2) → (3,2,-2)
(3,1,6) → (1,3,6)
(3,4,9) → (4,3,9)
```

La transposición conserva nnz.

27. Transposición con conteo por columnas

Fundamento

Para producir la transpuesta ordenada por fila puede contarse cuántas entradas tiene cada columna original. Las sumas prefijas indican el inicio de cada fila resultante.

Después se coloca cada entrada en su segmento de destino y se avanza su posición de escritura.

Ejemplo y aplicación

Para A, cada columna tiene una entrada:
Conteos = [1,1,1,1,1].
Inicios = [0,1,2,3,4,5].

Con dimensiones y entradas conocidas, el proceso de conteo y distribución cuesta O(c + z), sin incluir recorridos de validación ni pasos adicionales de normalización.

28. Suma de matrices dispersas

Fundamento

La suma C = A + B requiere que ambas matrices tengan las mismas dimensiones. Para cada coordenada se suman los valores correspondientes; una entrada ausente aporta cero.

Si el resultado de una coordenada es cero, puede omitirse de la representación normalizada.

Ejemplo y aplicación

Sea B, también de 4 × 5:
B[0,2] = -5
B[1,1] = 4
B[3,4] = 1

Entonces C[0,2] = 0 desaparece.
C[1,1] = 4 se incorpora.
C[3,4] = 10 reemplaza la suma.
Las demás entradas de A se conservan.

29. Suma mediante fusión ordenada

Fundamento

Si ambas listas de entradas están ordenadas por fila y columna, pueden combinarse con dos cursores.

Se copia la menor coordenada. Si coinciden, se suman sus valores y se avanza en ambas listas. Se omiten resultados iguales a cero.

Ejemplo y aplicación

A: (0,2,5), (1,0,8), (2,3,-2), (3,1,6), (3,4,9).

B: (0,2,-5), (1,1,4), (3,4,1).

C: (1,0,8), (1,1,4), (2,3,-2), (3,1,6), (3,4,10).

Costo de fusión: O(zA + zB), sin ordenar las entradas ni inicializar cabeceras.

30. Multiplicación por un escalar

Fundamento

Para calcular B = λA se multiplica cada valor almacenado por λ. Las coordenadas se mantienen si el resultado continúa siendo distinto de cero.

Si λ = 0, la matriz resultante es nula y puede representarse sin entradas activas, conservando sus dimensiones.

Ejemplo y aplicación

Con λ = 2, los valores de A pasan de:
[5,8,-2,6,9]
a [10,16,-4,12,18].

Las dimensiones siguen siendo 4 × 5.

Con aritmética de punto flotante, cualquier regla de eliminación por tolerancia debe ser explícita: no equivale a comprobar cero exacto.

31. Producto de matriz por vector

Fundamento

Si A tiene r × c y x tiene c componentes, y = Ax tiene r componentes.

Cada salida y[i] acumula A[i,j] × x[j] sobre las entradas no nulas de la fila. CSR facilita este recorrido sin revisar todos los ceros de la matriz.

Ejemplo y aplicación

Para x = [1,2,3,4,5]:
y[0] = 5×3 = 15.
y[1] = 8×1 = 8.
y[2] = -2×4 = -8.
y[3] = 6×2 + 9×5 = 57.

Resultado: y = [15,8,-8,57].
Costo usual en CSR: O(r + z).

32. Pseudocódigo de producto en CSR

Fundamento

Los límites de row_ptr evitan inspeccionar entradas de otras filas. El vector resultado se inicializa con r ceros.

Para cada fila se utilizan únicamente los valores y columnas de su segmento, multiplicando por la componente correspondiente del vector de entrada.

Pseudocódigo

```
matvec(val,col,ptr,x,r):
  y = vector_ceros(r)
  para i = 0 hasta r-1:
    para k = ptr[i] hasta ptr[i+1]-1:
      y[i] += val[k] * x[col[k]]
  retornar y

Requisito: longitud de x = c.
Fila vacía: su salida permanece cero.
```

33. Producto de dos matrices dispersas

Fundamento

Para A de r × k y B de k × c, el producto C tiene r × c. Cada entrada C[i,j] suma productos A[i,t] × B[t,j].

El costo depende de cuántas contribuciones se generan, de cómo se acumulan y del patrón de ambas matrices. No se reduce automáticamente a O(zA + zB).

Ejemplo y aplicación

Advertencia estructural: el resultado puede ser denso.

Una matriz columna de n unos tiene n no nulos. Una matriz fila de n unos también tiene n.

Su producto exterior es una matriz n × n de unos, con n^2 no nulos. La dispersión de las entradas no garantiza la del resultado.

34. Resumen de complejidades

Operación	Representación / supuesto	Costo	Observación
Acceso A[i,j]	COO sin índice	O(z)	Recorre entradas
Acceso A[i,j]	CSR con columnas ordenadas	O(log(zi+1))	zi: entradas en fila i
Insertar entrada	CSR en arreglos compactos	O(z+r) peor caso	Desplaza y ajusta
Suma	COO ordenado y normalizado	O(zA+zB)	Solo fusión
Producto Ax	CSR	O(r+z)	Inicializa y recorre

No existe una complejidad única de «matriz dispersa». Deben indicarse el formato, la organización y la operación.

35. Aplicaciones: grafos y conexiones

Fundamento

Una matriz de adyacencia registra conexiones entre vértices. En grafos con pocas aristas respecto al máximo posible, muchas posiciones no contienen conexión.

Una representación dispersa permite guardar las conexiones existentes y recorrer los vecinos de un vértice por su fila.

Ejemplo y aplicación

Si hay V vértices y E aristas dirigidas sin entradas repetidas, una representación por filas necesita O(V + E) espacio.

Si una arista puede tener peso cero, «ausencia» y «peso cero» deben distinguirse mediante metadatos o una convención adicional: cero implícito no basta.

36. Aplicaciones: datos y cálculo científico

Fundamento

En una matriz usuario-producto, una entrada puede registrar una interacción observada. En una matriz documento-término, puede representar la frecuencia de una palabra.

En simulación numérica, las relaciones locales entre puntos suelen generar sistemas con muchas entradas nulas.

Ejemplo y aplicación

No confundir cero con desconocido.

La ausencia de una valoración no siempre significa que el usuario haya dado una puntuación cero. La aplicación debe representar correctamente los datos faltantes.

El formato se elige según la construcción, el acceso por filas o columnas y las operaciones posteriores.

37. Validación y errores frecuentes

Fundamento

Comprobar dimensiones, límites de índices, alineación de arreglos y coherencia de punteros. En CSR, el último puntero debe coincidir con la cantidad almacenada.

En una estructura ortogonal, cada nodo debe poder encontrarse tanto desde su fila como desde su columna.

Ejemplo y aplicación

Casos de prueba:
- Matriz sin entradas.
- Filas y columnas vacías.
- Índices en los límites.
- Coordenadas repetidas.
- Actualización y asignación cero.
- Suma con cancelación.
- Doble transposición: $(A^T)^T = A$.
- Comparación con un cálculo denso pequeño.

38. Ejercicio guiado de modificación

Fundamento

Partir de A, la matriz base 4 × 5. Aplicar estas operaciones en orden:

1. Insertar A[1,4] = 7.
2. Actualizar A[0,2] = 12.
3. Asignar A[2,3] = 0.
4. Consultar A[2,3].
5. Construir CSR del resultado.

Ejemplo y aplicación

```
Entradas finales:
(0,2,12), (1,0,8), (1,4,7), (3,1,6),
(3,4,9).

Consulta A[2,3]: 0 implícito.
val = [12,8,7,6,9]
col = [2,0,4,1,4]
row_ptr = [0,1,3,3,5]

La fila 2 queda vacía; nnz final = 5.
```

39. Ejercicio de cálculo y trabajo solidario

Fundamento

Continuar con la matriz modificada del ejercicio anterior y usar x = [1,2,3,4,5].

En parejas, una persona interpreta row_ptr y otra calcula los productos; luego intercambian funciones para revisar el resultado. Cada integrante explica una fila del cálculo.

Ejemplo y aplicación

Resultado esperado:
y[0] = 12×3 = 36.
y[1] = 8×1 + 7×5 = 43.
y[2] = 0.
y[3] = 6×2 + 9×5 = 57.

y = [36,43,0,57].

Cierre: explicar cómo ayudar al compañero a detectar un índice incorrecto sin resolverle toda la actividad.

Conceptos clave aprendidos

Cero implícito y valor almacenado
Dimensiones, coordenadas y nnz
Densidad y ahorro de memoria
Tripletas, COO, CSR y CSC

Listas por filas y estructura ortogonal
Inserción, actualización y eliminación
Transposición, suma y producto
Solidaridad al explicar y revisar

Referencias

Almacenamiento y análisis

Netlib. Templates for the Solution of Linear Systems. Almacenamiento de matrices dispersas.

https://netlib.org/linalg/html_templates/node91.html

Formatos e implementación

SciPy. Documentación oficial de estructuras dispersas y formatos COO / CSR.

https://docs.scipy.org/doc/scipy/tutorial/sparse.html

Recursos y práctica

Lectura complementaria

NIST / Matrix Market: formato de coordenadas para intercambiar matrices.

https://math.nist.gov/MatrixMarket/formats.html

Atención: Matrix Market usa índices base uno; convertirlos antes de usar los ejemplos base cero.

Aplicación

Actividad posterior:
Implementar set, get y transposición con tripletas o listas.

Comparar el resultado con una matriz densa pequeña. Registrar memoria estimada y costo de recorrido.

Documentar cómo se manejan duplicados, ceros y dimensiones.

PAUSA DE HIDRATACIÓN

DUDAS

¿Por qué guardar las dimensiones?
¿Cuándo conviene CSR?
¿Cómo se elimina un nodo ortogonal?
¿Cómo aplicamos la solidaridad?